IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Received 21 January 2025, accepted 29 January 2025, date of publication 3 February 2025, date of current version 6 February 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3537660

APPLIED RESEARCH

Real-Time Analysis of Event Response Based on RT-Thread

HUI LIU[1], WENYIN TAO[1], ZHIXIN HONG[2], AND LIANMIN SHI[2]
[1]School of Intelligent Management, Suzhou Industrial Park Institute of Services Outsourcing, Suzhou 215123, China
[2]Suzhou Xiyu Information Technology Company Ltd., Suzhou 215168, China

Corresponding author: Hui Liu (71814695@qq.com)

This work was supported in part by Jiangsu Vocational Education Integrated Circuit Technology "Double-Qualified" Master Studio under Grant Su Teacher[2022] No.13, and in part by Suzhou Science and Technology Project "Development and Industrialization of Secure and Authenticated IoT SoC Module" under Grant ZXL2022025.

ABSTRACT The event response is a crucial feature in the real-time operating system RT-Thread, used for synchronizing threads. This paper aims to analyze the real-time performance of the event response and provide practical insights for its application. To achieve this, a low-latency piling timing analysis approach is proposed. The precise measurement of thread scheduling time is achieved by accurately recording timestamp of critical moments without external interruptions. Moreover, the theoretical time of thread scheduling under different compilation optimization levels is analyzed. And, a thread synchronization experiment based on event response is designed to measure the actual time of thread scheduling on Arm Cortex-M4 microcontroller STM32L431RCT6. The experimental results show that the low-latency piling timing analysis approach proposed in this paper has an error of about 1.5 microseconds compared to the measurement time of the logic analyzer and the theoretical analysis time. Finally, by comparing and analyzing theoretical time and actual measurement time of thread scheduling, it is evident that the compilation optimization level and PendSV exception are the key factors influencing the real-time performance of RT-Thread event response.

INDEX TERMS RT-thread, event response, piling timing analysis, compilation optimization level, PendSV.

I. INTRODUCTION

The RT-Thread (Real Time-Thread) is an open-source RTOS (Real Time Operating System) designed for embedded and IoT applications [1], [2]. It supports multiple threads and offers various methods for thread synchronization, including events, semaphores, mutexes, message queues, and priority-based thread scheduling [3], [4]. Currently, RT-Thread has a strong software and hardware ecosystem, with a large community of developers and widespread use in areas such as wireless transmission, agricultural production, environmental monitoring, motor control, and robotics [5].

As an open-source real-time operating system, RT-Thread has been extensively studied by many scholars in terms of its operating mechanism and loading approach on different architecture chips. For example, the startup process of RT-Thread has been thoroughly analyzed. The loading approach of RT-Thread on ARM Cortex-M MCUs has also been well studied [6], [7]. By loading RT-Thread in the on-chip bootloader [8], physical isolation between RT-Thread and user applications can be effectively achieved, reducing program compile time and improving development efficiency. The key elements of deploying RT-Thread on processors with different architectures have been deeply analyzed, and the implementation of RT-Thread deployment on RISC-V architecture application processor D1-H has been successfully achieved [9], [10], [11]. For applications with strict real-time requirements, such as those requiring millisecond or even microsecond precision, real-time performance is a crucial factor to consider when choosing an

The associate editor coordinating the review of this manuscript and approving it for publication was Liang-Bi Chen.

© 2025 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

RTOS. It is essential to have a deep understanding of the real-time performance of RTOS thread scheduling and data communication [12].

Real-time performance refers to the ability of an RTOS to respond within a specified time. Previous research on the real-time performance of RTOS has primarily focused on the overall performance of the system, including context switching for thread scheduling, interrupt context switching [13], [14], and semaphore scheduling strategies [15]. However, there is a lack of intensive quantitative analysis on the real-time performance between thread synchronization. Moreover, there is also a lack of experimental verification on the impact of different compilation optimization levels on the real-time of event response. In addition, using logic analyzers and other instruments for real-time measurement also has certain limitations, it is difficult to capture detailed temporal information of kernel level events, and flexibility of measurement are not high [16], [17], [18].

Therefore, this paper aims to investigate the events in the synchronization and communication mechanism of RT-Thread. It will use a low-latency piling timing analysis approach to understand the operation mechanism of events and identify key technical factors that affect the real-time response of events. The ultimate goal is to improve the real-time performance of RT-Thread and promote its widespread use [19].

In this paper, the event response in RT-Thread will be explained first, followed by a proposed low-latency piling timing analysis approach for real-time analysis [20]. Next, a verification experiment for real-time analysis of event responses was designed and implemented on a hardware platform using the STM32L431RCT6 MCU. Finally, the experimental results are analyzed from both theoretical and practical perspectives. The key factors affecting the real-time performance of RT-Thread event response under different optimization levels are identified, as well as the influence of the kernel exception called Pendable System Vector (PendSV) on real-time performance.

II. EVENT MECHANISM IN RT-THREAD

Synchronization mechanisms often schedule threads in a multi-threaded RTOS to complete specific tasks or avoid conflicting access to shared resources [21], [22]. The synchronization mechanisms in RT-Thread mainly include events, semaphores, mutexes, message queues, etc [23], [24]. Events are often used as synchronization approaches when coordinating threads but do not require data transmission.

Event control blocks typically represent events, which contain information like event type, status, and task list waiting for events. The object control block is the source of the event control block in the RT-Thread kernel. If events are used to synchronize threads, there must be at least one thread in the waiting state. Other threads or interrupts can notify whether the event has been completed, thereby waking up the thread waiting on it and changing its state from Wait to Ready.

In RT-Thread, the event control block consists of thread waiting lists and a 32-bit word known as the event word, as shown in Figure 1. Each bit in the event word represents an independent event that does not interfere with others. The default state is 0, indicating that the event has not occurred. A value of 1 indicates that the event has occurred, and the RT-Thread kernel will actively wake up the relevant threads waiting for that event bit.

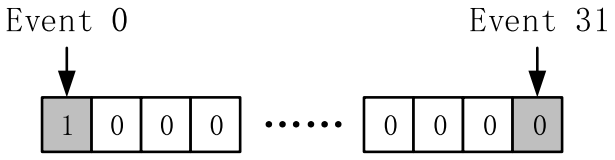


FIGURE 1. 32-bit event word.

RT-Thread has three main functions related to events: (1) rt_event_create(), which is used to create event objects, allocate memory space, set blocking wakeup mode, and initialize the event word to 0. (2) rt_event_send(), which sets the corresponding event bit in the event word, queries and removes the threads related to the set event bits in the blocking queue, loads them into the ready queue, and then schedules the thread with the highest priority. (3) rt_event_recv(), which determines if the events that threads are waiting for have occurred. If the events have occurred, they will be returned directly. If they have not occurred, the current thread will be placed in the blocking queue and the timer will be activated. At the same time, the scheduler will search the ready queue to find the thread with the highest priority and run it.

The function rt_event_recv() is typically invoked by the controlled thread, which enters a blocking queue and waits for an event to occur. On the other hand, rt_event_send() is usually called by interrupts or other threads. When one or more event bits in the event word are set, the threads waiting for these events will exit the blocking state and become eligible for scheduling.

This paper examines the real-time response mechanism for RT-Thread events on ARM Cortex-M4 microcontrollers. These microcontrollers have a system kernel exception called PendSV, which is specifically designed for thread scheduling and context switching. The event function triggers thread scheduling by setting the 28th bit, PENDSVSET, in the interrupt control and status register (ICSR). Typically, PendSV is given the lowest priority to ensure the real-time capability of the RTOS and to prioritize external interrupts. The PendSV service routine is responsible for updating the stack pointer (SP) and program counter (PC) and performing context switching after external interrupts have been executed. The execution process of PendSV is illustrated in Figure 2.

PendSV can improve the stability of an RTOS and prevent tasks from being unable to switch. However, it also adds complexity to real-time analysis. This paper proposes a low-latency piling timing analysis method to accurately assess

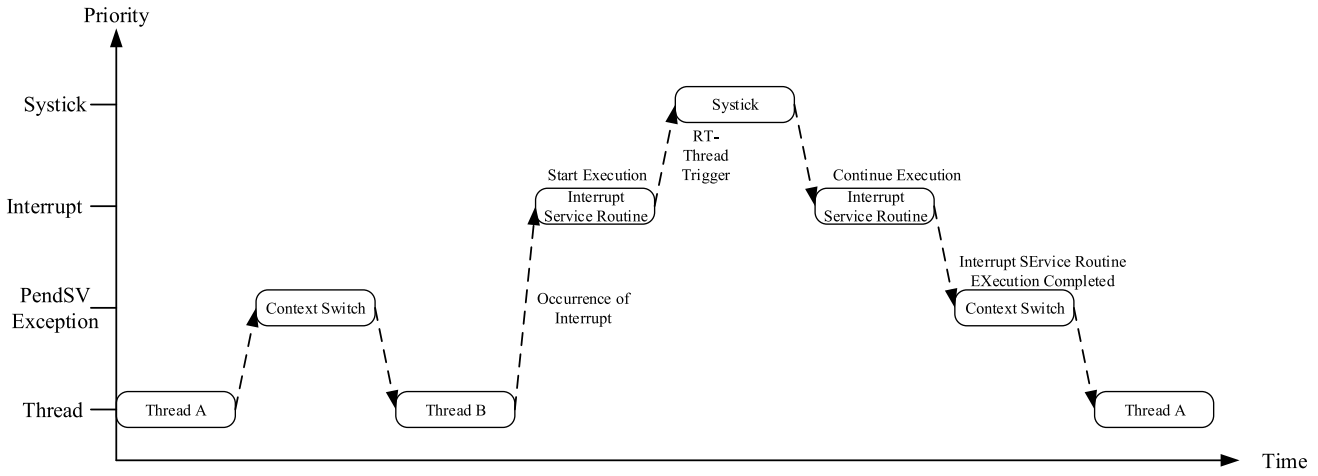


FIGURE 2. The execution mechanism of PendSV.

the real-time performance of the event response mechanism when external interrupts are blocked.

III. A LOW-LATENCY PILING TIMING ANALYSIS APPROACH

Timing analysis is a widely used method for examining the dynamic execution of embedded programs. This approach involves identifying specific breakpoints within the program, at which key information is output for analysis [25]. The typical outputs in timing analysis include oscilloscope readings and pile-driving debugging.

In order to accurately assess the real-time capability of an RTOS using oscilloscopes, it is necessary to select a test pin (GPIO pin) and reverse its state at two specific timing analysis points. This allows for the measurement of program execution time by observing the duration of the voltage level between the two points. While using oscilloscopes for this purpose provides high accuracy, the testing process itself is quite complex and requires testers to have a strong understanding of oscilloscope operation and precise control over the inversion of GPIO pin states. Alternatively, pile driving debugging involves setting breakpoints in embedded programs and using printf() to display important data for evaluating the real-time performance of the RTOS. This method is simpler and more intuitive compared to using oscilloscopes.

The real-time capability of printf() is inadequate. For instance, when using a microcontroller with a kernel clock frequency of 48MHz and a serial port baud rate of 115200, printf() takes an average of 8.68 us to output a single character. This is significantly longer than the RTOS scheduling time. If printf() is used to output information during critical programs for analyzing the real-time capability of the RTOS, it can lead to significant errors [26].

To prevent errors caused by the printf() function, the StartTime and EndTime global variables are utilized to store the clock count values at the beginning and end of the test code, respectively. The difference between these two values is then printed using the printf() function at the end of the test code. As the global variables are measured in nanoseconds, their impact on the accuracy of the test results can be disregarded, resulting in a low latency approach for analyzing pile-driving timing.

When utilizing the low-latency piling timing analysis method proposed in this paper, it is important to take note of the following technical points:

1 Timer initialization. Since the main thread (main()) only executes once, the timer used for analyzing timing must be initialized and started in the main(). Not only does this approach complete initialization before the timer is used, but it also prevents errors from being caused by repeated initialization of the timer.

2 Selection of Timer Clock Reference: When initializing the timer, it is necessary to set its prescaler value in order to determine the reference clock for the counting cycle. For instance, the STM32L431RCT6 MCU has a kernel clock frequency of 48MHz and a system timer called SysTick that generates a timed 'tick' to serve as the clock reference for the entire system. The SysTick is a 24-bit downward counter and, when the SysTick clock source is set as the kernel clock without frequency division, each count represents 1/48 microseconds. This level of accuracy in counting time is sufficient for real-time analysis of RT-Thread event response [27].

3 Selection of testing points. The testing points for timing analysis include the start and the end testing point. There shouldn't be too much code between two testing points, because the timer count value is limited. If the code is too much, it may cause the running time to exceed the timer count limit. Then, there is a risk of overflow in the timer count value at the start testing point or end testing point, Therefore, it is impossible to accurately calculate the running time interval between the two testing points. Moreover, to further improve the accuracy of timing analysis, the selection of testing points must be close to the testing code. For timing analysis of RT-Thread

event response, the start testing point needs to be set in the sentence before calling the send event function rt_event_send(), and the end testing point needs to be set in the sentence after calling the receive event function rt_event_rcv().

4 Calculation of code execution time. Once the timer has been initialized and the clock reference has been established, it is possible to directly read the value in its counting register to obtain the clock count value of the testing point. For example, the Systick timer module of STM32L431RCT6 MCU has four 32-bit registers, namely the control and status register CTRL, overload register LOAD, counter VAL, and calibration value register CALIB. Among them, the counter SysTick->VAL stores the current clock count value, and the testing code can directly obtain the values of SysTick->VAL and separately assign them to the global variables StartTime and EndTime to calculate the code execution time T_{ce}. The calculation formula of T_{ce} for upward counting timers without overflow is as follows:

$$T_{ce} = (EndTime - StartTime) \div F_{mcu} \quad (1)$$

The calculation formula of T_{ce} for downward counting timers without overflow is as follows.

$$T_{ce} = (StartTime - EndTime) \div F_{mcu} \quad (2)$$

F_{mcu} represents the kernel clock frequency of the MCU, and this paper is based on a downward counting timer [28].

IV. TIMING ANALYSIS OF EVENT RESPONSE

The General Embedded Computer(GEC) based on Arm Cortex-M microcontrollers developed by the Embedded Laboratory of Soochow University is utilized as the hardware platform for timing analysis, as depicted in Figure 3. The accompanying IDE (Integrated Development Environment) is Keil uVision V5.38.0.0 with Arm Compiler V5.06 update 7 (build 960).

The GEC utilizes the STM32L431RCT6 MCU as its core microcontroller, with a maximum kernel clock frequency of 48MHz. It also has 256K Flash memory, primarily used for storing program code and interrupt vector tables, and 64K SRAM, mainly used for global variables, static variables, and the stack. These specifications are sufficient for meeting the operational needs of RT-Thread.

A. DESIGN OF TESTING EXPERIMENT

Communication between the blue light thread and red light thread is achieved through events in the testing experiment. After the RT-Thread starts, a blue light thread (thd_bluelight) and a red light thread (thd_redlight) are created. In thd_bluelight, the rt_event_send() function is used to set the red light event bit in the event word. In thd_redlight, the thread is suspended and waits for the red light event bit to be set by calling rt_event_recv(). Once the red light event bit is set, the thread reverses the state of the red light. Additionally, the timer count values stored in the global variables StartTime and EndTime during the event response scheduling process are used to calculate the scheduling time and are outputted through printf().

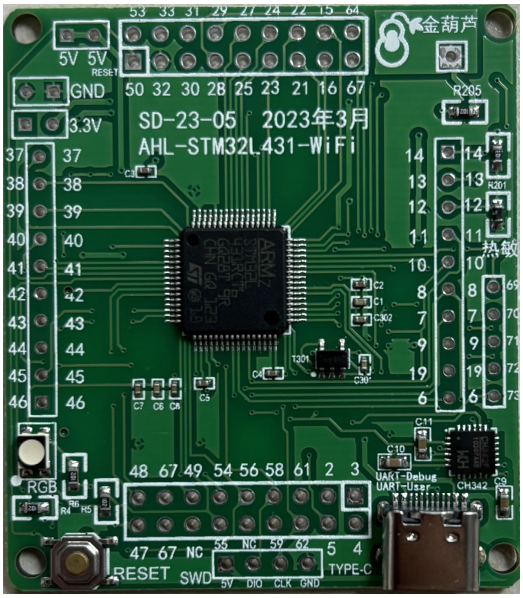


FIGURE 3. General Embedded Computer(GEC).

The program structure of the testing experiment is as follows.

```
/*1.Define global variables
    StartTime and EndTime in the
    includes.h*/
G_VAR_PREFIX vuint32_t StartTime;
G_VAR_PREFIX vuint32_t EndTime;
/*2.Create event word, blue light
    and red light threads in the
    main thread file
    threadauto_oppinit.c*/
//(1)Declare variables
#define RED_LIGHT_EVENT (1<<3)
rt_thread_t thd_redlight;
rt_thread_t thd_bluelight;
struct rt_event *EventWord;
//(2)Create event word
EventWord =
    rt_event_create("EventWord",
    IPC_FLAG_PRIO);
//(3)Create blue light and red light
    thread
thd_bluelight =
    rt_thread_create("bluelight",
    (void *)thread_bluelight, 0,
    512, 10, 35);
thd_redlight =
    thread_create("redlight", (void
```

```
    *)thread_redlight, 0, 512, 10,
    35);
//(4)Startup blue light and red
    light thread
thread_startup(thd_bluelight);
thread_startup(thd_redlight);
/*3.blue light thread:Set red light
    thread event bit of the event
    word*/
StartTime = SysTick->VAL;
rt_event_send(EventWord,
    RED_LIGHT_EVENT);
/*4.red light thread:Waiting for the
    set of red light thread event
    bit, Calculate scheduling time*/
rt_uint32_t recvedstate;
rt_event_recv(EventWord,
    RED_LIGHT_EVENT,
    EVENT_FLAG_OR|EVENT_FLAG_CLEAR,
WAITING_FOREVER,&recvedstate);
EndTime = SysTick->VAL;
if(SysTick_HasOverflowed){
  //Scheduling time calculation in
      case of overflow
  printf("Scheduling time =%d\r\n",
      StartTime-Endtime+
      SysTick->Load);
}
else{
  // Scheduling time calculation in
      case of non-overflow
  printf("Scheduling time =%d\r\n",
      StartTime{-} EndTime);
}
```

The execution process of the test experiment is shown in Figure 4.

B. TIMING ANALYSIS

In RT-Thread, the thread scheduling process based on events is as follows. Once threads and events are created, the threads waiting for the event bits are placed in the block queue. When the event bits are set, the corresponding threads are retrieved from the block queue and moved to the ready queue. Their status is then changed to "running". This paper aims to analyze the time it takes for the event bit to be set and the thread to switch to the running state, in order to investigate the key factors that influence the real-time performance of RT-Thread event response and verify the effectiveness of the low-latency piling timing analysis approach in terms of measurement accuracy.

1) THEORETICAL TIME FOR THREAD SCHEDULING

The theoretical time of thread scheduling can be obtained by calculating the total execution time of compiled machine instructions, while in the Cortex-M4 processor architecture, the cycles of different machine instructions vary. In the Cortex-M4 processor technical reference manual provided by ARM, the cycles of machine instructions are listed, some of which are shown in Table 1.

TABLE 1. ARM Cortex-M4 processor instruction set summary(Segment).

Operation	Description	Assembler	Cycles
Move	Register	MOV Rd, <op2>	1
	16-bit immediate	MOVW Rd, #<imm>	1
	To PC	MOV PC, Rm	1+P
Add	Add	ADD Rd, Rn, <op2>	1
	Add to PC	ADD PC,PC, Rm	1+P
	Form address	ADD Rd, <Label>	1
Subtract	Subtract	SUB Rd, Rn, <op2>	1
	Subtract with borrow	SBC Rd, Rn, <op2>	1
Multiply	Multiply	MUL Rd, Rn, Rm	1
	Multiply accumulate	MLA Rd, Rn, Rm	1
	Long signed	SMULL RdLo,RdHi,Rn,Rm	1
Divide	Signed	SDIV Rd, Rn, Rm	2-12
	Unsigned	UDIV Rd, Rn, Rm	2-12
Compare	Compare	CMP Rn, <op2>	1
	Negative	CMN Rn, <op2>	1
Logical	AND	AND Rd, Rn, <op2>	1
	Exclusive OR	EOR Rd, Rn, <op2>	1
	OR	ORR Rd, Rn, <op2>	1
Shift	Logical shift left	LSL Rd, Rn,#<imm>	1
	Arithmetic shift right	ASR Rd, Rn, #<imm>	1
Rotate	Rotate right	ROR Rd, Rn, Rs	1
Count	Leading zeroes	CLZ Rd, Rn	1
Load	Halfword	LDRH Rd, [Rn, <op2>]	2
	Doubleword	LDRD Rd, Rd, [Rn, #<imm>]	1+N
Store	Halfword	STRH Rd, [Rn, <op2>]	2
	Doubleword	STRD Rd, Rd, [Rn, #<imm>]	1+N
Push	Push	PUSH <reglist>	1+N
	Push with link register	PUSH <reglist>, LR	1+N
Pop	Pop	Pop <reglist>	1+N
	Pop and return	POP <reglist>, PC	1+N+P

Among them, (1) P represents the number of cycles required for a pipeline refill. The value ranges from 1 to 3 depending on the alignment and width of the target instruction, as well as whether the processor manages to speculate the address early. (2) N represents the number of registers to be loaded or stored in the register list, including PC or LR. (3) After the division operation is completed, it terminates, and the required execution cycle depends on the value of the input operand. Moreover, division operation is capable of being interrupted, which means that it can be temporarily paused, with a worst-case delay of one execution cycle, and resumed after the interruption is finished.

The compiled machine instructions for the experimental program described in this paper can be found in the *.axf file. In RT-Thread, thread scheduling during event response involves three main stages: sending event words (rt_event_send), scheduling (rt_schedule), and the PendSV exception. Therefore, the machine instructions in these three stages are crucial for analyzing the theoretical time of thread scheduling. However, the number of instructions in these

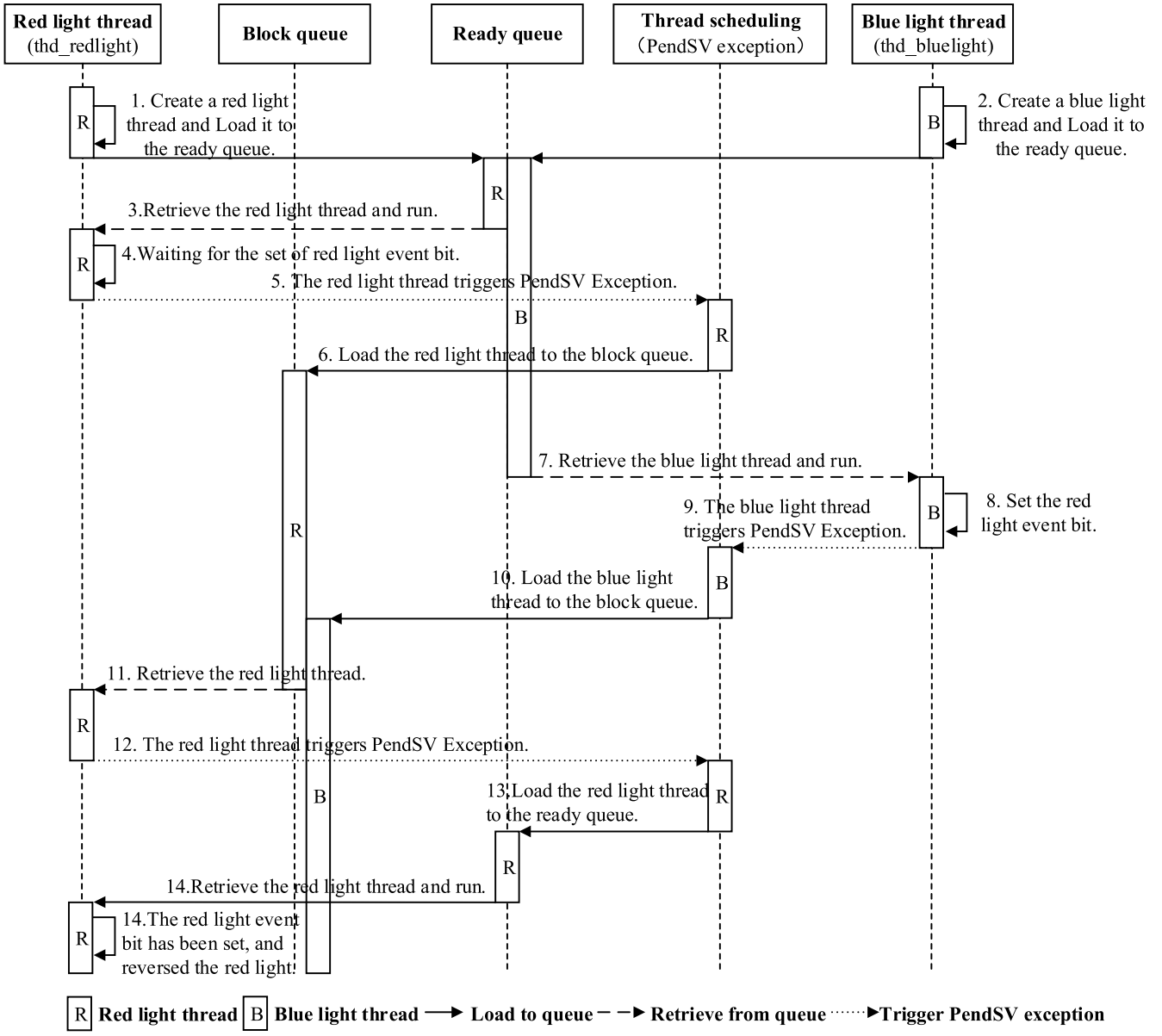


FIGURE 4. The execution process of the test experiment.

stages varies depending on the compilation optimization level. To address this, experiments were conducted using different optimization levels, including Non-optimization (-O0), Basic Optimization (-O1), Intermediate Optimization (-O2), Advanced Optimization (-O3), and Optimization of removing influence debugging based on -O0 (-Og), as provided by IDE Keil uVision V5.38.0.0.

The number of event response scheduling instructions under different compilation optimization levels is shown in Table 2.

The execution cycle of a single instruction t_o is primarily determined by the kernel clock frequency of the MCU F_{mcu}, $t_o = 1/F_{mcu}$. In this paper, the STM32L431RCT6 is used with the kernel clock frequency of 48MHz, and the execution cycle of a single instruction is $t_o = 1/48\mu s$. Based on the number of instructions listed in Table 2, the theoretical scheduling time for event response under different compilation optimization levels is presented in Table 3.

TABLE 2. Number of event response scheduling instructions under different compilation optimization levels.

Stage	-O0	-O1	-O2	-O3
rt_event_send(μs)	473	386	363	327
rt_schedule(μs)	258	206	193	193
PendSV(μs)	116	116	116	116
Total(μs)	847	708	672	636

TABLE 3. The theoretical scheduling time for event response under different compilation optimization levels.

Compilation Optimization Levels	-O0	-O1	-O2	-O3
Theoretical Scheduling Time(μs)	17.64	14.75	14.00	13.25

In practical scenarios, the actual instruction execution time may differ from the theoretical value due to the use of a three-stage pipeline in ARM Cortex-M4 microcontrollers. While this pipeline is designed to complete each instruction within one clock cycle, encountering a branch instruction can cause the pipeline to be flushed, resulting in a longer execution time. Even when the branch instruction is not hit, it still takes an additional 2-3 cycles to refill the pipeline. Furthermore, accessing instructions and data stored in Flash or SRAM typically involves waiting for status, which can significantly increase the execution time.

2) ACTUAL TIME FOR THREAD SCHEDULING

The actual time taken for thread scheduling during event response can be determined using the formula 2. In the experimental testing program, two global variables, StartTime and EndTime, are defined to store the timer count values at the beginning and end of thread scheduling. The counting cycle is set to $1/48\mu s$ during timer initialization. When initializing threads, the priority of the red light thread will be set higher than that of the blue light thread. This ensures that the red light thread runs first and enters the state of waiting for the event. Subsequently, the blue light thread runs and sets the event word to avoid missing the reception of the event word. Before the blue light thread calls rt_event_send() to set the red light event bit, the current timer count value, SysTick->VAL, is assigned to StartTime and printed using printf(). The red light thread receives the red light event bit by calling rt_event_recv() and assigns the current timer count value, SysTick->VAL, to EndTime. This value is also printed using printf(). Finally, the thread scheduling time is calculated using the formula 2, and the values of StartTime and EndTime are printed by printf().

During the experiment, we connected the GEC to the computer using the TTL serial port to USB device. We then used a serial port tool to receive the actual time for thread scheduling, which was sent by the GEC through the printf() function, as shown in Figure 5.

The values from 50 experiments are averaged to obtain the actual scheduling time for event response under different compilation optimization levels, as shown in Table 4.

TABLE 4. The actual scheduling time for event response under different compilation optimization levels.

Compilation Optimization Levels	-O0	-O1	-O2	-O3
Actual Scheduling Time(μs)	19.19	16.33	15.56	14.77

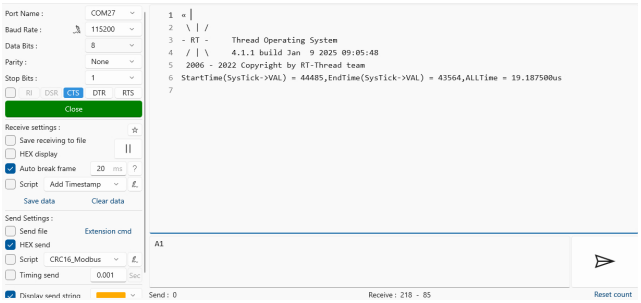


FIGURE 5. Thread Scheduling Information from the Serial Port Tool.

Using the logic analyzer to measure the thread scheduling time of event response by capturing the level inversion of GEC test pins. In the testing program, initialize the test pin to a high level, flip the pin to a low level before calling rt_event_send(), and flip the pin back to a high level after calling rt_event_recv(), The duration of low level is the thread scheduling time of event response. To improve the execution time of I/O pin level switching code and enhance measurement accuracy, it is essential to directly manipulate the register to control the pin level instead of relying on advanced library functions. The actual operation involves connecting the logic analyzer to a computer, setting the sampling accuracy to 100MHz, and connecting the GEC test pin to channel 0. The sampling results are shown in Figure 6.



FIGURE 6. Thread Scheduling Information from the Logic Analyzer.

The actual scheduling time of event response under different compilation optimization levels is obtained by the logic analyzer, as shown in Table 5.

TABLE 5. The actual scheduling time of event response under different compilation optimization levels through logic analyzer.

Compilation Optimization Levels	-O0	-O1	-O2	-O3
Actual Scheduling Time(μs)	18.35	16.04	15.79	14.73

After comparing Table 3, Table 4, and Table 5, it is evident that the actual scheduling time for RT-Thread event response closely aligns with the theoretical scheduling time. This demonstrates that increasing the compiler optimization level has a direct impact on the number of instructions, resulting in a shorter event response time. Additionally, when comparing Table 4 and Table 5, it is apparent that the low-latency piling timing analysis approach yields similar results to those obtained from a logic analyzer in terms of accuracy. The margin of error is also minimal, within a few tens of nanoseconds, and the discrepancy with the theoretical scheduling time is only 1.5 microseconds. Therefore, the low-latency piling timing analysis approach proposed in this paper

effectively verifies the real-time performance of RT-Thread event response.

C. FACTORS AFFECTING TIMING ANALYSIS OF RT-THREAD EVENT RESPONSE

After analyzing the theoretical and actual time of thread scheduling based on event response, it is evident that two main factors significantly impact the real-time performance of RT-Thread event response. Firstly, the number of instructions can be reduced through compilation optimization, resulting in shorter running time and improved real-time performance. Secondly, ARM Cortex-M4 MCUs utilize the PendSV exception to facilitate thread scheduling and context switching. However, the PendSV exception has the characteristic of 'suspended execution', meaning it must wait for all external interrupts to be executed before executing its own service routine and context switching. This can also affect the accuracy of timing analysis.

1) INFLUENCE OF COMPILATION OPTIMIZATION

By analyzing the experimental results of RT-Thread event response scheduling under different compilation optimization levels, it is found that its real-time performance will be greatly affected by compilation optimization. At -O0 level, the scheduling time can be about 20 us, and after the optimization is turned on, it can be reduced by about 7 us, greatly improving its real-time performance.

In this paper, the focus is on compilation optimization, specifically in terms of the optimization level. The commonly used levels of compilation optimization are as follows: (a) No optimization (-O0), where the compiler does not optimize the code. (b) Basic optimization (-O1), where the compiler performs basic optimizations such as removing unused constants/variables and simplifying code branches. (c) Intermediate optimization (-O2), where the compiler optimizes at the register and instruction level, including function call graph optimization and wider range inlining. (d) Advanced optimization (-O3), which builds upon -O2 and utilizes pseudo register networks, automatic vectorization, and other optimizations. Other optimization levels include -Og, which removes debugging influence based on -O0, and -Os, which optimizes for code size based on -O2 [29]. Generally, the higher the optimization level, the better the real-time event response in RT-Thread, similar to other RTOSs like mBed OS [30].

2) INFLUENCE OF PendSV EXCEPTION

PendSV exception is a unique software interrupt in ARM Cortex-M MCUs. It is triggered by code and is set as the lowest priority. Therefore, it will be suspended due to other interrupts' preemption, and its own service routine will not be executed until the execution of other interrupts is completed, and it is mainly used to implement context switching.

When RT-Thread is deployed to run on ARM Cortex-M MCUs, thread scheduling is achieved through PendSV exceptions to switch the context of tasks, which will prolong thread scheduling time when external interrupts are triggered, resulting in reduced real-time performance. Therefore, when using RT-Thread to develop strong real-time applications, it is required that hardware interrupts should be as few as possible and interrupt service programs should be as simple as possible.

In addition, to enhance the real-time performance of thread scheduling, RT-Thread utilizes assembly language instructions for writing PendSV exceptions.

V. CONCLUSION

In RT-Thread, events are primarily used for thread synchronization. However, they have a wider range of applications compared to semaphores. They can not only achieve one-to-one thread synchronization, but also one-to-many and many-to-many synchronization. This paper focuses on researching the real-time performance of RT-Thread event response. Firstly, the event response in RT-Thread is explained, followed by a proposed low-latency piling timing analysis approach. This approach is used to analyze the real-time performance of RT-Thread event response. Next, a thread synchronization experiment is designed and implemented to analyze the theoretical scheduling time of event response under different compilation optimization levels. The actual thread scheduling time is also measured. The results indicate that there is a discrepancy of approximately 1.5 microseconds between the actual scheduling time of event response and the theoretical analysis time. This approach can achieve accuracy similar to that of a logic analyzer, demonstrating the effectiveness of the proposed low-latency piling timing analysis method in accurately evaluating the real-time performance of RT-Thread event response without the need for external interrupts. Finally, the key factors affecting the real-time performance of RT-Thread event response are analyzed, with a conclusion that the main factors include compilation optimization level and PendSV exception.

REFERENCES

[1] K. C. Wang, *Embedded and Real-Time Operating Systems*, 2nd ed., Cham, Switzerland: Springer, 2023, doi: 10.1007/978-3-031-28701-5.

[2] D. Mamone, A. Bosio, A. Savino, S. Hamdioui, and M. Rebaudengo, "On the analysis of real-time operating system reliability in embedded systems," in *Proc. IEEE Int. Symp. Defect Fault Tolerance VLSI Nanotechnol. Syst. (DFT)*, Frascati, Italy, L. Dilillo, M. Psarakis, and T. Siddiqua, Eds., Oct. 2020, pp. 1–6, doi: 10.1109/DFT50435.2020.9250861.

[3] G. Buttazzo, *Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications*, 4th ed., Cham, Switzerland: Springer, 2024, doi: 10.1007/978-3-031-45410-3.

[4] A. N. Godunov, I. I. Khomenkov, V. G. Shchepkov, and A. V. Khoroshilov, "Configurable test system for RTOS," *Program. Comput. Softw.*, vol. 49, no. 4, pp. 286–294, Aug. 2023, doi: 10.1134/s0361768823040059.

[5] C. Zhong, H. Zhang, Y. Yuan, S. Zhang, and H. Wan, "Design and implementation of distributed time-triggered system based on RT-thread," in *Proc. 7th Int. Conf. Inf. Technology: IoT Smart City*, Shanghai, China, Dec. 2019, pp. 472–476, doi: 10.1145/3377170.3377268.

[6] R.-G. Chang and T.-V. Hoang, "Constructing an AI compiler for ARM cortex-M devices," *Comput. Syst. Sci. Eng.*, vol. 46, no. 1, pp. 999–1019, 2023, doi: 10.32604/csse.2023.034672.

[7] M.-S. Chen and T. Chou, "Classic McEliece on the ARM cortex-M4," in *Proc. IACR Cryptol. ePrint Arch.*, 2021, p. 492. [Online]. Available: https://eprint.iacr.org/2021/492

[8] H. C. A. van Tilborg and S. Jajodia, Eds., "BIOS basic input output system," in *Encyclopedia of Cryptography and Security*, 2nd ed., Cham, Switzerland: Springer, 2011, p. 147, doi: 10.1007/978-1-4419-5906-5_1324.

[9] S. P. Singhal, M. Sridevi, N. S. Narayanan, and M. J. S. Raman, "Porting of eChronos RTOS on RISC-V architecture," 2019, *arXiv:1908.11648*.

[10] P. Bulic, *Understanding Computer Organization: A Guide to Principles Across RISC-V, ARM Cortex, and Intel Architectures* (Undergraduate Topics in Computer Science). Cham, Switzerland: Springer, 2024, doi: 10.1007/978-3-031-58075-8.

[11] K.-D. Nguyen, T.-K. Dang, B. Kieu-Do-Nguyen, C.-K. Pham, and T.-T. Hoang, "RISC-V-based system-on-chips for IoT applications," in *Proc. IEEE Hot Chips 36 Symp. (HCS)*, Stanford, CA, USA, Aug. 2024, p. 1, doi: 10.1109/HCS61935.2024.10665181.

[12] S. Kim, J. Park, B. Park, S. Park, D. Park, and C. Jo, "An analysis of thread scheduling in mobile device using thread interaction graph," in *Proc. IEEE Int. Conf. Consum. Electron. (ICCE)*, Las Vegas, NV, USA, Jan. 2024, pp. 1–4, doi: 10.1109/ICCE59016.2024.10444212.

[13] R. Balas, A. Ottaviano, and L. Benini, "CV32RT: Enabling fast interrupt and context switching for RISC-V microcontrollers," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 32, no. 6, pp. 1032–1044, Jun. 2024, doi: 10.1109/TVLSI.2024.3377130.

[14] J. Kotker, D. Sadigh, and S. A. Seshia, "Timing analysis of interrupt-driven programs under context bounds," in *Proc. Int. Conf. Formal Methods Comput.-Aided Design (FMCAD)*, Austin, TX, USA, P. Bjesse and A. Slobodová, Eds., Oct. 2011, pp. 81–90. [Online]. Available: http://dl.acm.org/citation.cfm?id=2157670

[15] X. Jiang, N. Guan, W. Liu, and M. Yang, "Scheduling and analysis of parallel real-time tasks with semaphores," in *Proc. 56th ACM/IEEE Design Autom. Conf. (DAC)*, Las Vegas, NV, USA, Jun. 2019, pp. 1–6, doi: 10.1145/3316781.3317907.

[16] M. L. Scott and T. Brown, *Shared-Memory Synchronization*, 2nd ed., Cham, Switzerland: Springer, 2024, doi: 10.1007/978-3-031-38684-8.

[17] I. Haur, J.-L. Béchennec, and O. Roux, "Formal verification of the inter-core synchronization of a multi-core RTOS kernel," in *Proc. 23rd Int. Conf. Formal Eng. Methods (ICFEM)*, in Lecture Notes in Computer Science, vol. 13478, Madrid, Spain, A. Riesco and M. Zhang, Eds., Cham, Switzerland: Springer, Jan. 2022, pp. 140–155, doi: 10.1007/978-3-031-17244-1_9.

[18] I. C. Bertolotti and G. G. Z. Kashani, "On the performance of open-source RTOS synchronization primitives," in *Proc. IEEE 1st Int. Forum Res. Technol. Soc. Ind. Leveraging Better Tomorrow (RTSI)*, Turin, Italy, Sep. 2015, pp. 398–402, doi: 10.1109/RTSI.2015.7325131.

[19] S. Rinaldi, A. Depari, P. Ferrari, A. Flammini, A. Musatti, and E. Sisinni, "Accurate time synchronization in a low-cost platform for the design of a distributed digital events detector," in *Proc. IEEE Int. Instrum. Meas. Technol. Conf. (I2MTC)*, Kuala Lumpur, Malaysia, May 2023, pp. 1–6, doi: 10.1109/I2MTC53148.2023.10175883.

[20] F. Lekschas, X. Zhou, W. Chen, N. Gehlenborg, B. Bach, and H. Pfister, "A generic framework and library for exploration of small multiples through interactive piling," *IEEE Trans. Vis. Comput. Graphics*, vol. 27, no. 2, pp. 358–368, Feb. 2021, doi: 10.1109/TVCG.2020.3028948.

[21] D. B. de Oliveira, R. S. de Oliveira, and T. Cucinotta, "A thread synchronization model for the PREEMPT_RT Linux kernel," *J. Syst. Archit.*, vol. 107, Aug. 2020, Art. no. 101729, doi: 10.1016/j.sysarc.2020.101729.

[22] J. Guo, Z. Li, C. Shi, and R. Zhao, "Thread scheduling sequence generation based on all synchronization pair coverage criteria," *Int. J. Softw. Eng. Knowl. Eng.*, vol. 30, no. 1, pp. 97–118, Jan. 2020, doi: 10.1142/s0218194020500059.

[23] A. Segu and W. Golab, "Recycling memory in recoverable mutex locks," in *Proc. 10th Int. Conf. Networked Syst. (NETYS)*, in Lecture Notes in Computer Science, vol. 13464, M. Koulali and M. Mezini, Eds., Cham, Switzerland: Springer, Jan. 2022, pp. 31–36, doi: 10.1007/978-3-031-17436-0_3.

[24] Z. Wei, J. Liu, Z. Sheng-ju, Z. Hu, and X. Zhao, "An LSTM-based method for message queue throughput prediction," in *Proc. 5th Int. Conf. Algorithms, Comput. Artif. Intell. (ACAI)*, Sanya, China, Dec. 2022, pp. 1–6, doi: 10.1145/3579654.3579724.

[25] M. Matraszek, M. Banaszek, W. Ciszewski, and K. Iwanicki, "Franken-Trace: Low-cost, cycle-level, widely applicable program execution tracing for ARM cortex-M SoC," in *Proc. Cyber-Phys. Syst. Internet Things Week*, San Antonio, TX, USA, May 2023, pp. 72–77, doi: 10.1145/3576914.3587521.

[26] P. Sivaranjani, S. Sasikala, A. Lavanya, and M. Keerthana, "Design and analysis of UART protocol with SEC-DED and implementation on FPGA," in *Proc. 14th Int. Conf. Comput. Commun. Netw. Technol. (ICCCNT)*, Delhi, India, Jul. 2023, pp. 1–8, doi: 10.1109/ICCCNT56998.2023.10306744.

[27] N. Gandraß, M. Rottleuthner, and T. C. Schmidt, "UTimer: A uniform low-level timer API for RIOT OS," in *Proc. IEEE Int. Conf. Internet Things (iThings) IEEE Green Comput. Commun. (GreenCom) IEEE Cyber, Phys. Social Comput. (CPSCom) IEEE Smart Data (SmartData) IEEE Congr. Cybermatics (Cybermatics)*, Melbourne, VIC, Australia, Dec. 2021, pp. 1–8, doi: 10.1109/iThings-GreenCom-CPSCom-SmartData-Cybermatics53846.2021.00016.

[28] T. Kiriyama, Y. Wu, and S. Yamane, "Reduction of timer interrupts for embedded assembly programs based on reduction of interrupt handler executions," in *Proc. IEEE 10th Global Conf. Consum. Electron. (GCCE)*, Kyoto, Japan, Oct. 2021, pp. 464–466, doi: 10.1109/GCCE53005.2021.9622013.

[29] Y. Ni, X. Du, Y. Yuan, R. Xiao, and G. Chen, "Tsoa: A two-stage optimization approach for GCC compilation options to minimize execution time," *Automated Softw. Eng.*, vol. 31, no. 2, p. 39, Nov. 2024, doi: 10.1007/s10515-024-00437-w.

[30] C. Bock, M. Marquardt, A. Martens, O. Simanski, and O. Hagendorf, "Smart sensors and actors with BACnet^{TM} and Mbed OS on cortex-M microcontrollers," in *Proc. IEEE 5th World Forum Internet Things (WF-IoT)*, Limerick, Ireland, Apr. 2019, pp. 937–942, doi: 10.1109/WF-IoT.2019.8767341.



HUI LIU received the M.S. degree from the School of Computer Science and Technology, Soochow University, Suzhou, China, in 2007. His research interests include embedded systems, deep learning, and embedded artificial intelligence.



WENYIN TAO received the M.S. degree in 2011. He is currently pursuing the Ph.D. degree with the School of Computer Science and Technology, Nanjing University of Aeronautics and Astronautics (NUAA), Nanjing, China. His research interests include computer vision, deep learning, and image processing.



ZHIXIN HONG received the B.S. degree from the School of Mathematics and Computer Science, Wuyi University, Wuyishan, China, in 2022. His research interests include the Internet of Things and embedded artificial intelligence.



LIANMIN SHI received the Ph.D. degree from the School of Computer Science and Technology, Soochow University, Suzhou, China, in 2019. His research interests include the Internet of Things and embedded artificial intelligence.